Electric Power Systems Research 243 (2025) 111478



Contents lists available at ScienceDirect

Electric Power Systems Research

journal homepage: www.elsevier.com/locate/epsr





Short-term electric load forecasting based on series decomposition and Meta-Informer algorithm

Lianbing Li [a], Xingchen Guo [a,*], Ruixiong Jing [b]

[a] School of Electrical Engineering, Hebei University of Technology, Tianjin, 300130, China
[b] School of Artificial Intelligence and Data Science, Hebei University of Technology, Tianjin, 300130, China

ARTICLE INFO

Keywords:
Load forecasting
Sequence decomposition
Meta-learning
Informer
BiGRU
Clustering

ABSTRACT

Accurate electricity load forecasting is crucial for the effective development of power management strategies. However, achieving both forecasting accuracy and efficiency is often challenging. This paper presents a novel framework that integrates Seasonal-Trend decomposition using Loess (STL), clustering, and meta-learning for electricity load forecasting. First, the local regression-based STL method decomposes the electricity load data into trend, seasonal, and residual components. Next, data slicing and clustering are performed based on seasonal and residual patterns. Using the Local Search k-means++ with Foresight(FLS++) clustering method, we expand the clustered data to generate multiple training tasks, which are then trained using the meta-learning-based Meta-Informer forecasting model. Subsequently, we assess the smoothness of the seasonal and residual testing tasks using the Standard Differenced Smoothness (SDS) metric. Adaptive filtering processes the data, and the model is fine-tuned for accurate predictions. Additionally, we employ a BiGRU model to forecast the trend component, which is summed and reconstructed to yield the final prediction results. Experimental results demonstrate that our approach effectively enhances forecasting accuracy in electricity load prediction.

1. Introduction

As global energy demand continues to grow, the importance of electricity load forecasting becomes increasingly pronounced. Accurate load forecasting not only aids in the efficient scheduling of energy resources within power systems but also enhances the safety and economic viability of grid operations [1]. In recent years, the application of deep learning and artificial intelligence methods in electricity load forecasting has garnered extensive research attention [2,3]. However, due to the complex temporal characteristics and uncertainties inherent in electricity load data, effectively improving both forecasting accuracy and efficiency remains a focal point of interest for both academia and industry.

Traditional time series models, such as the autoregressive integrated moving average (ARIMA) [4] model and other classical time series analysis methods [5], have achieved some success in short-term load forecasting. However, these methods are limited in their ability to handle nonlinear and complex time series data. To address this issue, neural network-based models, including artificial neural networks (ANN) [6], long short-term memory (LSTM) [7], gated recurrent units (GRU) [8], and convolutional neural network (CNN) ensembles [9,10], have been widely applied to load forecasting. Despite their strengths, these models struggle to capture long-range global dependencies. The Transformer model, with its self-attention mechanism, effectively captures global dependencies in data. However, its quadratic computational complexity with increasing sequence length limits its application in large-scale load forecasting. Consequently, various improved Transformer models have been developed. The Informer model [11], for instance, introduces an efficient sparse self-attention mechanism that significantly reduces computational complexity, making it a powerful tool for long-term sequence prediction. Moreover, several Transformer-based models [12,13] have demonstrated promising performance in time series forecasting. Nonetheless, single neural network models exhibit limitations when addressing the seasonality, trends, and randomness inherent in power load data. As a result, many researchers have combined heuristic algorithms with neural networks to enhance prediction accuracy [14,15]. For example, Xie et al. [16] employed particle swarm optimization (PSO) to optimize an evolutionary neural network (ENN), achieving optimal parameter solutions. However, the high computational cost and the risk of local optima hinder the efficiency of the model. Similarly, Dai and Yu [17], along with Fan et al. [18], utilized Bayesian Optimization (BO) to optimize their proposed models, while Zhang and Hong [19] adopted an improved grey wolf optimization algorithm (CGWO) to find optimal parameters. Despite their effectiveness, these

* Corresponding author.
E-mail address: guowork2025@163.com (X. Guo).

https://doi.org/10.1016/j.epsr.2025.111478
Received 25 October 2024; Received in revised form 28 January 2025; Accepted 29 January 2025
Available online 8 February 2025
0378-7796/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Nomenclature

ANN	Artificial neural network
ARIMA	Auto regressive integrated moving average
BiGRU	Bidirectional gated recurrent unit
CGWO	Chaotic grey wolf optimizer
CNN	Convolutional neural network
EMD	Empirical mode decomposition
ENN	Elman neural network
FCM	Fuzzy c-means
FLS++	Local search k-means++ with foresight
GRU	Gated recurrent unit
ICEEMDAN	Intrinsic computing expressive empirical mode decomposition with adaptive noise
ICSS	Iterative cumulative sum of squares
LSTM	Long short-term memory
PCA	Principal component analysis
PSO	Particle swarm optimization
STL	Seasonal and trend decomposition using loess
SVDE	Secondary variation differential evolution
SVM	Support vector machine
SVR	Support vector regression
VMD	Variational mode decomposition
WGRP	Weighted gray relationship projection

methods are time-intensive and only capable of optimizing external parameters, leading to inefficiencies in the training and prediction processes.

To address these challenges and further improve prediction accuracy, researchers have explored the integration of time series decomposition techniques with deep learning methods. For example, Dai et al. [20] combined stationary wavelet decomposition (SWD) with adaptive neuro-fuzzy inference systems (ANFIS), but their model suffered from sensitivity to initial values. Ma et al. [21] employed variational mode decomposition (VMD) alongside Mogrifier-LSTM for forecasting, yet encountered difficulties in setting VMD parameters. Similarly, Çevik et al. [22] utilized empirical mode decomposition (EMD) with support vector machines (SVM), but the mode mixing in EMD and SVM's inability to capture temporal dependencies limited prediction accuracy. Li et al. [23] improved prediction accuracy using complementary ensemble empirical mode decomposition with adaptive noise (CEEMDAN), although at the expense of high computational complexity. Further advancements were made by Li et al. [24], who combined improved CEEMDAN (ICEEMDAN) with bidirectional GRU, significantly enhancing short-term load forecasting performance. Likewise, Huang et al. [25] proposed a hybrid EMD-LSTM model for building cooling load forecasting. However, these decomposition techniques often lead to increased computational costs and model complexity without providing adequate solutions. To mitigate this, we propose a pretraining-based approach.

Although many studies have demonstrated that time series decomposition methods improve prediction accuracy, they often require separate predictions for each decomposed component, leaving room for efficiency improvements. Zha et al. [26] applied the seasonal-trend decomposition using LOESS (STL) for initial decomposition, followed by ICEEMDAN for further residual decomposition. However, the high spatial complexity of this approach significantly impacted model training and prediction speed. To address this, they employed an improved hierarchical clustering technique to group similar components, reducing prediction complexity. Nevertheless, there remains potential for further improvement in prediction efficiency.

To enhance the stability and adaptability of power load forecasting, some studies have introduced meta-learning [27,28], proving its effectiveness in improving prediction accuracy. Xiao et al. [29] proposed a Meta-ANN model, which further improved the accuracy and robustness of power load forecasting. Similarly, Sarmas et al. [30] combined meta-learning with LSTM to enhance prediction performance. However, integrating meta-learning with the Informer model is more effective in capturing global dependencies, thereby further improving prediction accuracy. Despite these advancements, many studies have overlooked the importance of time series decomposition in handling long- and short-term dependencies and the interactions among different seasonal and trend components.

We have summarized the methods used in existing references and their evaluations in Table 1. This study further integrates meta-learning with deep learning, with the main contributions and innovations as follows:

- A method for extracting meta-learning training tasks is proposed. This method segments the training data corresponding to the seasonal and residual components into multiple slices based on the slicing size. The FLS++ algorithm is then employed to cluster the sliced data, and the clustering results are flattened to generate training tasks for meta-learning.
- A curve smoothness evaluation metric was proposed and used as the basis for adaptive filtering.
- Additionally, a novel Meta-Informer algorithm is developed, integrating Reptile with an enhanced Informer model. This algorithm trains on the extracted meta-learning tasks and fine-tunes predictions using seasonal and residual test datasets, achieving improved forecasting efficiency while preserving prediction accuracy.

The remaining sections of this paper are organized as follows: Section 2 introduces the algorithm principles, Section 3 discusses the combined algorithm model, Section 4 presents experimental validation and comparative analysis, and Section 5 concludes the study.

2. Theoretical formulation of methods

2.1. STL

The STL is a time series decomposition method based on local regression smoothing techniques. It effectively separates a time series into three components: trend, seasonal, and residual [31].

$$y_t = T_t + S_t + R_t \tag{1}$$

In this context, y_t represents the observed value of the time series at time t, T_t denotes the trend component, S_t indicates the seasonal component, and R_t refers to the residual component.

The calculation methods for each component are as follows:

$$S_t = Loess(y_t, w_S) \tag{2}$$

$$T_t = Loess(y_t - S_t, w_T) \tag{3}$$

$$R_t = y_t - T_t - S_t \tag{4}$$

In this context, w_S and w_T represent the window widths for seasonal smoothing and trend smoothing, respectively.

Eqs. (2)–(4) can be divided into an inner loop and an outer loop. The inner loop serves as the core, while the outer loop iteratively calls the inner loop by fine-tuning the robust weights. The detailed process of the inner loop is shown in Fig. 1.

Here, T_0 represents the initial value of the trend component.

Table 1
Summary of existing load forecasting literature survey.

Reference	SD	ML	Efficient	Model	Forecasting Horizon	Measures	Year
[10]	×	×	✓	CNN-LSTM	30-min – 1-h – 3-h ahead	MAPE	2020
[16]	×	×	×	PSO-ENN	30-min ahead	RMSE, MAPE, MaxError, MinError	2020
[19]	✓	×	×	VMD-SVR-CGWO	1-h ahead	RMSE, MAPE, MAE, NRMSE	2021
[9]	×	×	✓	CNN-BiGRU-Attention	1-h ahead	RMSE, MAPE, MAE, R^2	2022
[29]	×	✓	✓	Meta-ANN	1-day ahead	MAPE	2022
[20]	✓	×	×	EMD-mRMR-WGRP-SVM-PSO	1-h ahead	MAPE, MAE, D_{state}	2022
[24]	✓	×	×	ICEEMDAN-GRA-SVDE-BiGRU-EC	15-min ahead	RMSE, MAPE, MAE, R, S	2023
[23]	✓	×	×	CEEMDAN-SE-LSTM	1-h ahead	RMSE, MAPE, MAE	2023
[8]	×	×	✓	CNN-GRU-Attention-iResNet	1-h ahead	RMSE, MAPE, Running time	2023
[7]	×	×	✓	PCA-FCM-LSTM	1-h ahead	MAPE	2024
[18]	✓	×	×	EWT-CNN-S-RNN + LSTM	30-min ahead	RMSE, MAPE, MAE, MSE, R^2	2024
[26]	✓	×	×	STL-CEEMDAN-SVR-GRU	30-min ahead	RMSE, MAPE, MAE, R^2	2024
[25]	✓	×	×	EMD-LSTM-Markov	15-min ahead	RMSE, MAPE, R^2	2024
[17]	✓	×	×	CNN-BO-TCN-Attention	1-h ahead	RMSE, MAPE, MAE, R^2	2024
[21]	✓	×	×	VMD-ICSS-hybrid neural network	1-h ahead	RMSE, MAPE, MAE, R^2	2025
Ours	✓	✓	✓	Series Decomposition and Meta-Informer	15-min ahead	RMSE, MAPE, MAE, MSE, MSPE	–

Note: **SD** Series Decomposition; **ML** Meta-Learning.

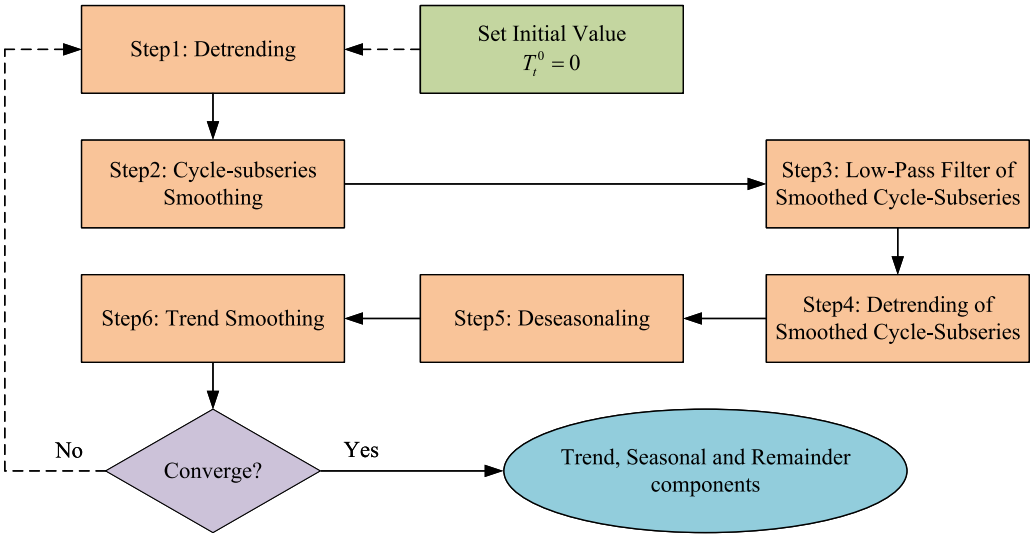


Fig. 1. The inner loop of STL decomposition.

2.2. Reptile

Reptile is an optimization algorithm used in meta-learning, designed to enhance a model's ability to quickly adapt to new tasks [32]. The core idea of the algorithm is to train across multiple tasks to obtain model parameters that can rapidly transfer to and adjust for new tasks. Specifically, Reptile learns a well-suited initial parameter θ by performing a few gradient updates on each task. For each task T_i, Reptile begins with the initial parameter θ and iterates multiple times according to the following update rule:

$$\theta' = \theta - \alpha \nabla_\theta \mathcal{L}\left(T_i; \theta\right) \tag{5}$$

In this context, α denotes the learning rate, $\mathcal{L}(T_i; \theta)$ represents the loss function for task T_i, and $\nabla_\theta \mathcal{L}\left(T_i; \theta\right)$ is the gradient of the loss function with respect to the parameter θ.

After completing the update for task T_i, Reptile uses the updated parameter θ' to calculate the new global parameter. Specifically, it averages the updates across all tasks to obtain the final parameter update:

$$\theta \leftarrow \theta + \beta\left(\theta' - \theta\right) \tag{6}$$

Here, β is a hyperparameter that controls the step size for the global parameter update. In this way, Reptile effectively aggregates experience from different tasks, enabling the model to quickly adapt to new tasks and thereby improving the efficiency of meta-learning.

In time series forecasting, traditional methods are typically designed for a specific task, and their robustness is relatively weak. Each training process starts from scratch, as indicated by the gray dashed arrow in Fig. 2. In contrast, Reptile consists of two stages: pretraining and fine-tuning. It uses the training set tasks for pretraining, and while the pretrained results may not be the optimal solution for each task, after fine-tuning on the test set tasks, the model can be quickly updated, thereby effectively improving prediction efficiency and accuracy. The process is illustrated by the solid arrow in Fig. 2. The model parameters are initialized randomly as θ_r, transitioned to meta-initialization as θ_m, and then fine-tuned to adapt to the target as θ_t.

2.3. FLS++

FLS++ is an optimization algorithm for k-means that combines local search with d^2 sampling, aimed at improving the effectiveness of k-means++ and LS++ [33]. The core idea of this algorithm is to more closely integrate local search with the Lloyd algorithm through a "predictive" mechanism. Specifically, FLS++ performs one round of the Lloyd algorithm at each center exchange, allowing for an early evaluation of the post-swap effect, rather than simply comparing the cost before and after the swap.

The operation of FLS++ begins with the selection of initial centers using d^2 sampling. Then, in each iteration, a new point, c_{new}, is chosen

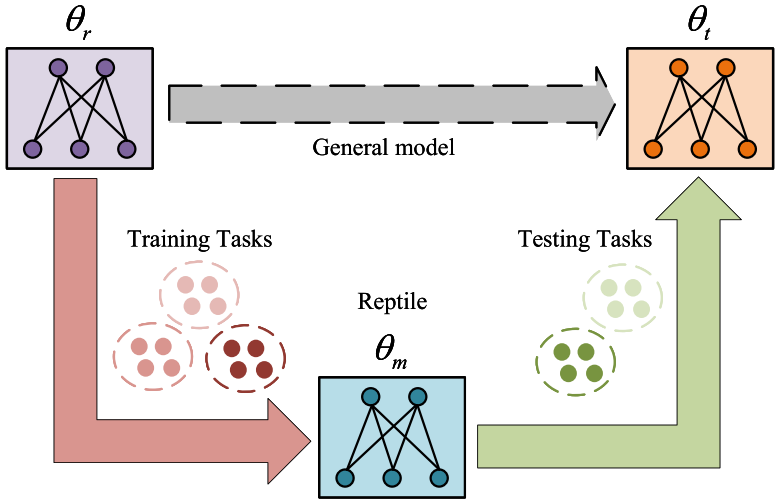


Fig. 2. Reptile Flowchart.

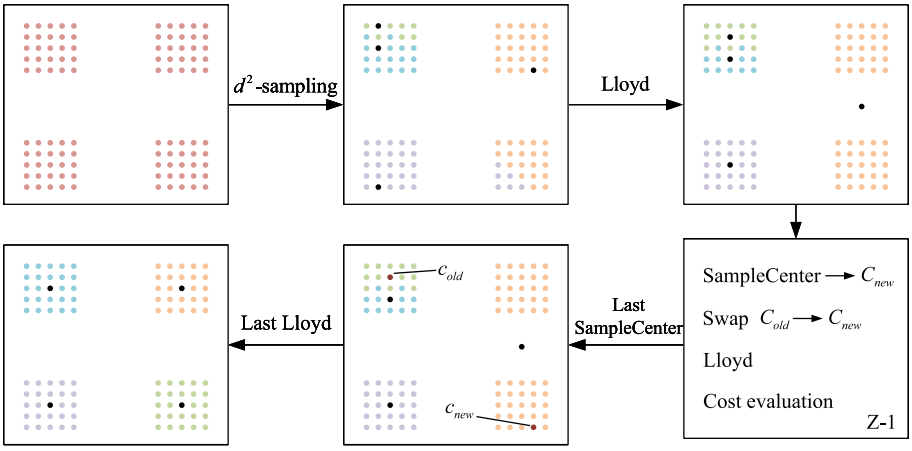


Fig. 3. FLS++ Flowchart.

through d^2 sampling and is considered as a replacement for an existing center, c_{old}. With each replacement, the algorithm executes one round of the Lloyd algorithm to calculate the cluster cost after updating the center. This process helps FLS++ better identify which exchanges will reduce the cost, particularly when removing redundant centers.

Moreover, FLS++ can be viewed as an enhanced version of LS++, with one round of the Lloyd algorithm executed between each local search step to prevent the solution from getting stuck in local optima. This strategy allows FLS++ to maintain a similar time complexity to LS++ while achieving higher solution quality across most datasets.

The detailed process is illustrated in Fig. 3, where each centroid replacement requires running one iteration of the Lloyd algorithm. The cost (sum of squared distances) after replacement is compared with the original cost. If the cost decreases, the replacement is retained. This step is repeated for a total of Z iterations.

3. Ensemble forecasting model

To improve the accuracy and efficiency of power load forecasting, this paper proposes a composite forecasting model based on sequence decomposition and Meta-Informer. The model design is grounded in a multi-scale feature analysis of power load time series, employing sequence decomposition, feature extraction, and clustering within a meta-learning framework to achieve precise modeling and forecasting of complex time-series data. The overall architecture comprises four key steps: sequence decomposition, data slicing and clustering, task construction and training, and prediction result fusion. This approach is aimed at effectively addressing the nonlinear and dynamic characteristics of power load demand.

To evaluate the predictive efficiency of the model, we introduced the measurement of computational time. For meta-learning, the meta-training phase is completed offline, while the meta-testing phase occurs online. Therefore, we only measure the time spent during the meta-testing phase, which includes training, validation, and testing during fine-tuning. For traditional testing models, to ensure consistency in timing data, we use the same dataset and timing method as in the meta-testing phase. The specific computational time is illustrated in Fig. 4.

3.1. Sequence decomposition

Power load time series typically exhibit complex characteristics, including trends, seasonality, and random fluctuations. To mitigate the impact of these characteristics on the model's predictive performance, the original load data is first decomposed. This study employs the STL decomposition method to break down the power load series into trend, seasonal, and residual components. The purpose of this process is to separate features across different time scales, thereby simplifying the complexity of subsequent predictive modeling.

The trend component captures the long-term changes in power load over time, represented by a relatively smooth curve that indicates the overall direction of load variation. The seasonal component reflects

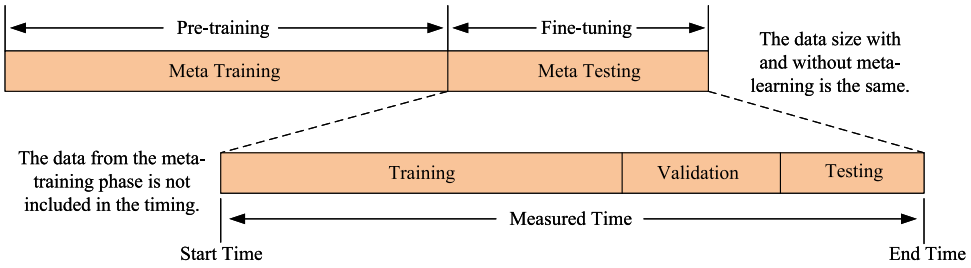


Fig. 4. Schematic diagram of prediction efficiency.

periodic changes in power load, often associated with daily cycles or seasonal fluctuations. In contrast, the residual component includes the random fluctuations, primarily composed of unpredictable anomalies or noise.

By employing this sequence decomposition approach, the model can effectively model different types of data, allowing the distinct dynamic features within the load time series to be separated and processed. This avoids the limitations of a single model that struggles to capture multiple dynamic patterns simultaneously.

3.2. Data slicing and clustering

To further enhance the model's ability to capture local features of the load data, this study performs slicing operations on the seasonal and residual components obtained from the decomposition. By implementing data slicing, long time series are divided into multiple short sub-sequences, allowing the model to better capture local fluctuations in load.

The slicing of the seasonal component primarily aims to extract periodic load fluctuation patterns. Each slice retains complete periodic information, ensuring that seasonal characteristics are adequately reflected in subsequent modeling. On the other hand, the slicing of the residual component extracts short-term fluctuations and random anomalies through smaller time windows, enabling the model to quickly respond to sudden load changes.

After slicing, the FLS++ clustering algorithm is employed to analyze the sliced sub-sequences. This algorithm groups similar load patterns into the same category, generating multiple clusters of typical load fluctuation patterns. The purpose of clustering is to identify potential patterns in the load data, thus providing a foundation for subsequent meta-learning tasks. Each cluster represents a typical load variation pattern, which helps improve the generalization capability of the meta-learning model when handling similar data.

3.3. Task construction and training

Based on the typical pattern clusters obtained through clustering, this study further constructs multiple training tasks and inputs them into the Meta-Informer meta-learning framework [34]. Meta-Informer is a meta-learning model specifically designed for sequence prediction, with the core goal of enhancing the model's adaptability to new tasks by sharing parameters and feature extraction networks across different tasks, thereby facilitating knowledge transfer.

Specifically, when performing electricity load forecasting, Meta-Informer initially employs the Informer model to extract key features from long sequential data and conduct preliminary predictions. Building on this foundation, the Reptile algorithm optimizes the model's global parameters through a multi-task learning mechanism, enabling the model to quickly adapt to data from different time periods, seasons, or load types.

During the meta-training phase, Meta-Informer first undergoes pretraining on all clustered tasks to learn shared feature representations across tasks. Once pretraining is completed, the model can quickly adjust its parameters based on new task data, enabling accurate load forecasting, as illustrated in Fig. 5. This meta-learning-based multi-task training approach not only improves the accuracy of power load forecasting but also enhances the model's adaptability to changes in load data distribution.

3.4. Generation and fusion of prediction results

After completing the task training, Meta-Informer predicts the seasonal and residual components separately. The inherent characteristics of these components dictate that the seasonal component is relatively smooth, whereas the residual component exhibits poorer smoothness. To address this, an adaptive filtering approach is proposed, which automatically smooths the curves based on their smoothness levels. Two smoothing methods are introduced: Savitzky-Golay filtering and Gaussian filtering.

By employing meta-learning, the time required for the traditional sequential decomposition of the trend and residual components is significantly reduced, allowing for only one round of model meta-training before fine-tuning the seasonal and residual data. This enhances the model's predictive efficiency. The trend component, being relatively smooth, is first standardized, followed by prediction using the BiGRU model, and subsequently inverse standardized. To achieve the final load forecasting results, this study adopts a fusion strategy that integrates the predictions of the seasonal, residual, and trend components, thereby improving the accuracy of electricity load predictions. The overall process is illustrated in Fig. 6.

3.5. Evaluation indicators

To better analyze and evaluate the accuracy of the load forecasting model, this study employs multiple evaluation metrics to comprehensively assess the model's predictive performance. These metrics include Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Percentage Error (MAPE), and Mean Squared Percentage Error (MSPE), which quantify the deviations between the predicted values and the actual load.

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}\left|y_i - \hat{y}_i\right| \tag{7}$$

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{y}_i\right)^2 \tag{8}$$

$$\mathrm{RMSE}=\sqrt{\mathrm{MSE}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{y}_i\right)^2} \tag{9}$$

$$\mathrm{MAPE} = \frac{1}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i + 10^{-10}}\right| \times 100 \tag{10}$$

$$\mathrm{MSPE} = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{y_i - \hat{y}_i}{y_i + 10^{-10}}\right)^2 \times 100 \tag{11}$$

In the equations, y_i represents the true value, $\hat{y}_i$ denotes the predicted value, and n is the number of data points. A small constant is added in both MAPE and MSPE to prevent division by zero errors.

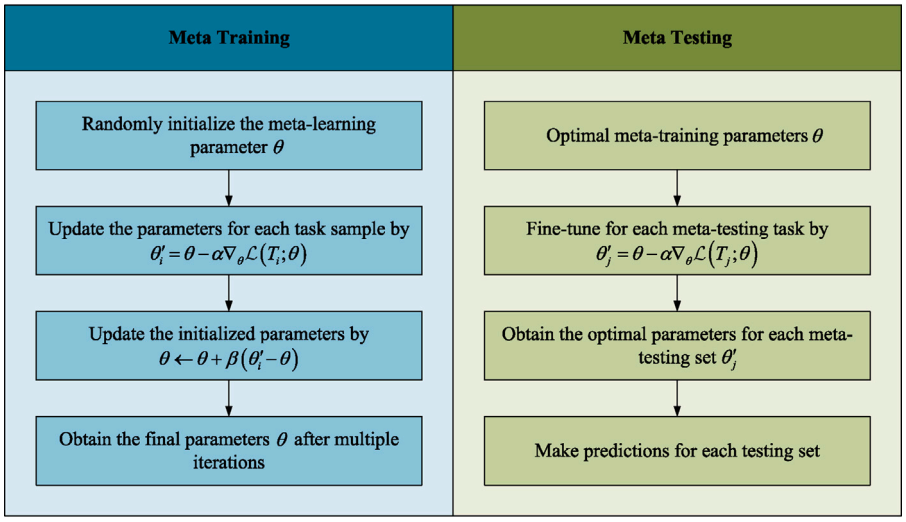


Fig. 5. Meta training And Meta testing.

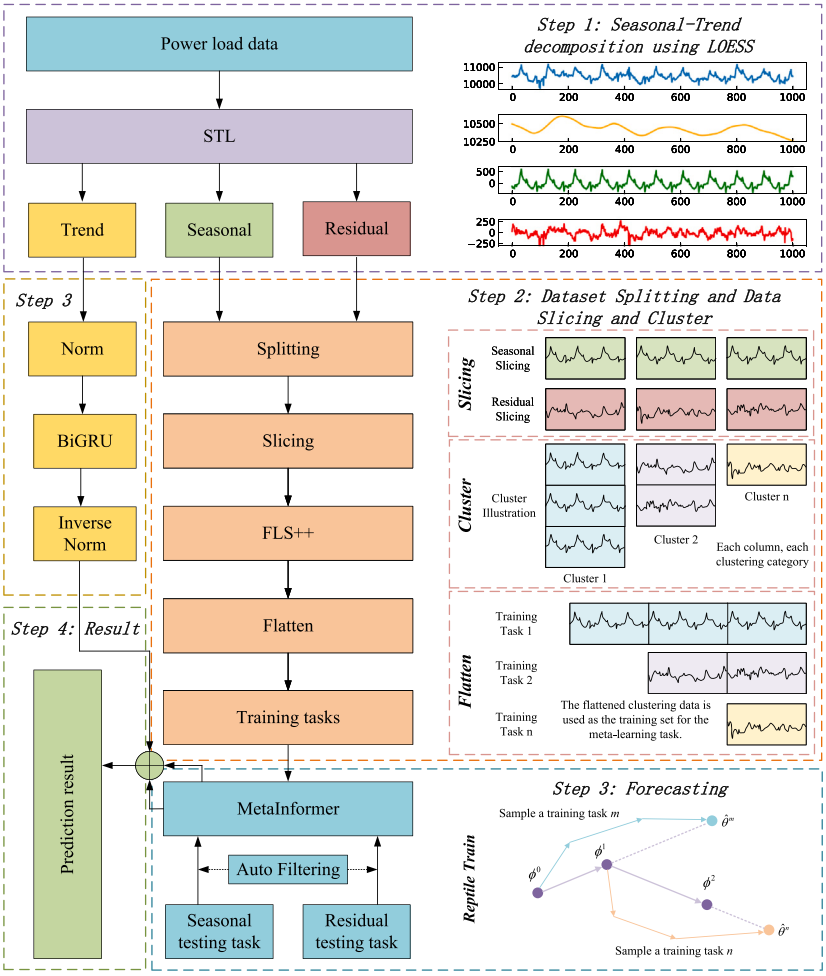


Fig. 6. Program block diagram.

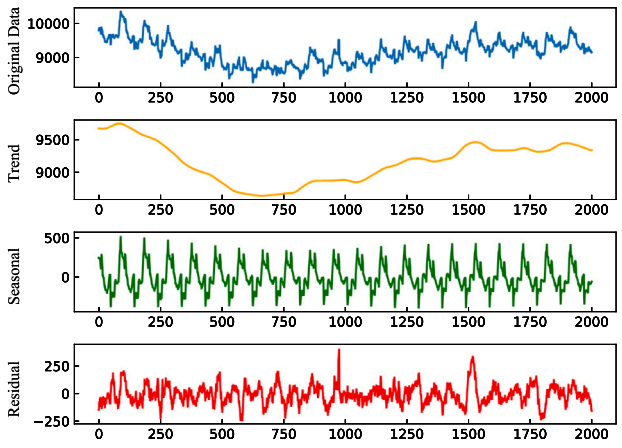


Fig. 7. STL decomposition results.

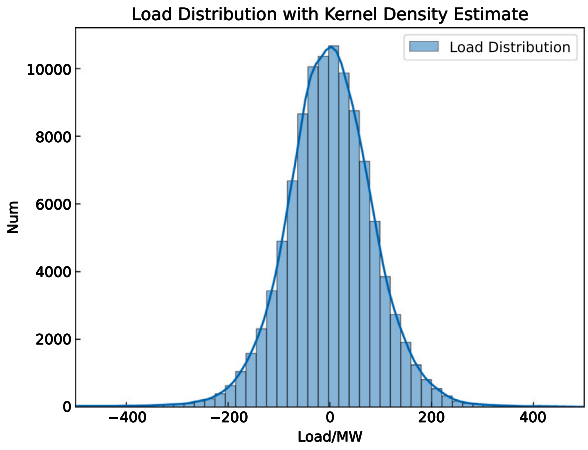


Fig. 8. Residual load distribution.

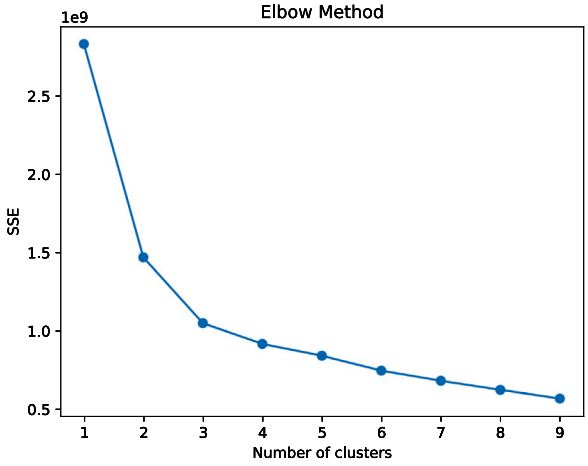


Fig. 9. Clustering elbow plots.

Furthermore, this study introduces the SDS as an innovative metric to quantify the smoothness of a curve, where a smaller SDS value indicates a smoother curve. For a set of curve data $y = [y_1, y_2, \ldots, y_n]$, the SDS is defined as follows:

$$\mathrm{SDS} = \sqrt{\frac{1}{n-1}\sum_{i=2}^{n}\left(\Delta y'_i - \overline{\Delta y'}\right)^2} \tag{12}$$

$$\Delta y'_i = y'_i - y'_{i-1}, i = 2, 3, \ldots, n \tag{13}$$

$$y'_i = \frac{y_i - \mu}{\sigma} \tag{14}$$

In this equation, μ and σ represent the mean and variance of the original data y, while $\overline{\Delta y'}$ denotes the mean of the standardized differenced data $\Delta y'_i$.

4. Results and discussions

The experimental data were sourced from a load dataset in a region of Ningxia, China, covering the period from January 1, 2016, to December 31, 2018, with a sampling interval of 15 min, resulting in a total of 105,216 data points. We employed a single-input single-output approach for forecasting, using historical load data to predict future load values. The prediction starts on December 10, 2018, and involves single-step forecasting. The meta-test dataset was divided into training, validation, and test sets in a ratio of 6:2:2.

The experimental hardware utilized included a 12th Gen Intel(R) Core(TM) i7-12650H processor and an NVIDIA GeForce RTX 4060 Laptop GPU.

4.1. The proposed model

To validate the proposed model, the aforementioned data were utilized as follows:

Step 1: STL Sequence Decomposition.

In the results section, we employed the STL decomposition from the statsmodels library, with the model version used in this study being [0.14.4]. Assuming a data sampling interval of 15 min, with 96 data points per day, the period parameter of the STL algorithm was set to 96, while all other parameters were configured to their default values. The decomposition results are shown in Fig. 7.

The trend component reflects the overall trajectory of the original electricity load data, while the seasonal component represents a more regular curve. Both components effectively reduce the difficulty of prediction. The residual components are visualized using kernel density estimation, as shown in Fig. 8, revealing a distribution that approximates a normal distribution with a mean of zero, indicating that the parameter settings are reasonable.

Step 2: Data Slicing and Clustering

We first divide the seasonal and residual components into training and testing tasks for meta-learning. The slicing size is determined based on the data points collected over a three-month period (one quarter), resulting in a slice size of 8,640. The last slice and any remaining data that do not form a complete slice are used to create the seasonal test task and residual test task for homogeneous task testing [34], while the remaining slices serve as the training set slices.

For all training set slices, we apply the FLS++ algorithm for clustering. To determine the appropriate number of clusters, we iterate through cluster counts in the range of $[1, 9]$ and use the "elbow method" to identify the optimal number of clusters. As shown in Fig. 9, the variance decreases rapidly when the number of clusters reaches 3, after which the rate of decrease slows down. This leads us to conclude that the optimal number of clusters is 3.

The clustered data is then flattened to match the time of each corresponding data point, forming a single input-single output time series data format.

Step 3: Task Construction and Training

The three sets of clustered data are used as the training sets for meta-learning, while the seasonal test task and residual test task serve as the testing sets. For the training, validation, and testing phases

Table 2
Hyperparameter table.

Model type	Hyperparameter	Value
Meta Model	Meta Train Epochs	10
	Meta Task Batch Size	2
	Meta Learning Rate	0.1
Basic Model	Train Epochs	200
	Batch Size	32
	Learning Rate	0.0001
	Input Sequence Length	96
	Start Token Length	48
	Prediction Sequence Length	1
	Encoder Input Size	1
	Decoder Input Size	1
	Model Dimension	256
	Number of Heads	8
	Number of Encoder Layers	2
	Number of Decoder Layers	1
	Fully Connected Network Dimension	1024
	Patience for Early Stopping	3
BiGRU Model	Train Epochs	20
	Batch Size	32
	Number of Hidden Layers	2
	Input Dimension	1
	Hidden Dimension	16
	Output Dimension	1

in meta-learning, we adopt a ratio of 6:2:2. Once the model has been trained during the meta-training phase, it only requires fitting and parameter tuning for the two corresponding testing datasets. This significantly enhances predictive efficiency and avoids the extensive training times associated with traditional models, which often operate primarily in offline prediction modes. Consequently, this approach provides a feasible solution for online predictions.

Step 4: Prediction Result Generation and Fusion

The hyperparameter settings used during the meta-learning training are presented in Table 2.

After training, the SDS is calculated for both the seasonal and residual components, yielding $SDS_S = 0.3004$ and $SDS_R = 0.5547$. Consequently, an adaptive filtering threshold of 0.4 is established, and the aforementioned parameter settings are employed for the predictions of both the seasonal and Residual components.

The Trend prediction is conducted using BiGRU, followed by summation and reconstruction to obtain the final prediction results.

4.2. Comparison of homogeneous algorithms

During the iterative improvement of the algorithm, we initially employed the Informer, which has a prediction efficiency with a complexity of $O(L \log L)$, to forecast the original data, L represents the length of the sequence. To enhance its prediction accuracy, we incorporated STL for sequence decomposition, thus constructing the STL-Informer model. Furthermore, to accelerate prediction efficiency and improve accuracy, we introduced meta-learning, developing the STL-based Meta-Informer model. Upon completing the meta-learning training, we fine-tuned the two decomposition components and implemented adaptive filtering, resulting in the establishment of the Meta-Informer-S and Meta-Informer-G models.

We conducted predictions for the seasonal and residual components to assess the model's efficiency, establishing four scenarios:

Scenario I: Train and predict the seasonal and residual components from the STL decomposition using the Informer, recording the total time required for both the training and testing phases.

Scenario II: Create a meta-training task set based on the STL-decomposed seasonal and residual components for model pre-training. During the meta-testing phase, we fine-tune the model and record the total time required for this phase.

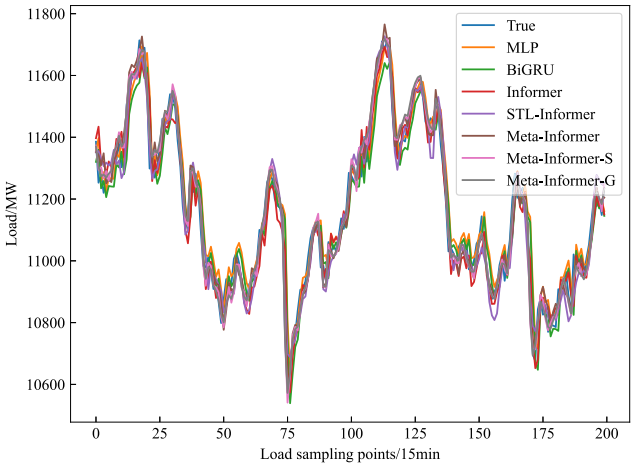


Fig. 10. Comparison diagram of homogeneous prediction algorithms.

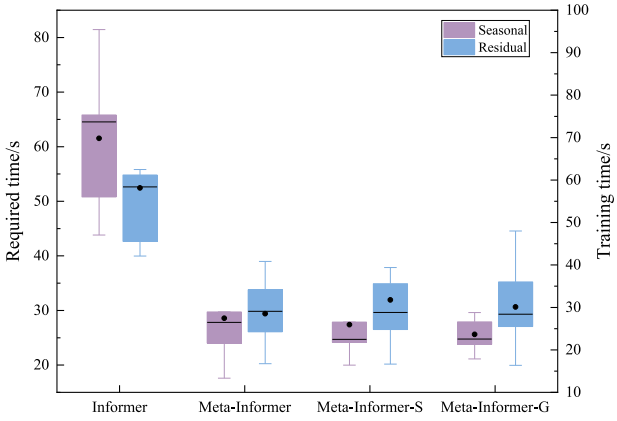


Fig. 11. Comparison of required time for homogeneous algorithms.

Scenario III: Building on Scenario II, introduce adaptive filtering during the meta-testing phase, employing Savitzky-Golay filtering.

Scenario IV: Also based on Scenario II, introduce adaptive filtering during the meta-testing phase, but using Gaussian filtering.

Each scenario underwent ten simulations.

For prediction accuracy, as shown in Table 3, Meta-Informer demonstrates a reduction of 14.214 in MAE and 0.128 in MAPE compared to Informer. This improvement is primarily attributed to the fact that, for measuring model efficiency, the same dataset was used as the control variable. However, through offline learning, meta-learning captures the historical features of the data, significantly enhancing prediction accuracy. Furthermore, for Meta-Informer-S and Meta-Informer-G in Scenario III and IV, the addition of filtering effectively removes data noise, enabling the model to better capture the main features of the data, thereby reducing RMSE to 36.494 and 35.440, respectively, which further improves the model's predictive performance (see Fig. 10).

As shown in Fig. 11, regardless of whether the seasonal or residual components were analyzed, the model efficiency utilizing meta-learning was significantly higher than that of the traditional model. For both the seasonal and residual components, the Meta-Informer reduced the average time required compared to the Informer by 53.55% and 50.94%, respectively. In Scenarios III and IV, there were no significant differences in the running time range or average and median required times compared to Scenario II. However, the Meta-Informer-G model in Scenario IV exhibited an increased upper limit in the time range for the residual component, with an average time that was only 5.27% longer than that of the Scenario II model.

Table 3
Homogenous algorithm error comparison.

Algorithm	MAE	MSE	RMSE	MAPE	MSPE
MLP	59.304	6375.582	79.847	0.536	0.0053
BiGRU	59.149	6155.996	78.460	0.534	0.0050
Informer	54.607	5012.556	70.799	0.493	0.0041
STL-Informer	47.253	3497.141	59.137	0.427	0.0029
Meta-Informer	40.393	2548.787	50.486	0.365	0.0021
Meta-Informer-S	29.014	1331.804	36.494	0.262	0.0011
Meta-Informer-G	28.247	1255.989	35.440	0.255	0.0010

Table 4
Comparison of different filtering types and algorithms.

Filter type	Algorithm	MAE	MSE	RMSE	MAPE	MSPE
None	Meta-Transformer	42.458	2831.060	53.208	0.383	0.0023
	Meta-Informer	40.393	2548.787	50.486	0.365	0.0021
	Meta-Reformer	38.860	2374.163	48.725	0.350	0.0019
	Meta-Autoformer	77.308	10 083.477	100.417	0.698	0.0082
	Meta-FEDformer	46.540	3581.277	59.844	0.420	0.0029
Savitzky Golay Filtering	Meta-Transformer-S	30.247	1468.336	38.319	0.273	0.0012
	Meta-Informer-S	**29.014**	**1331.804**	**36.494**	**0.262**	**0.0011**
	Meta-Reformer-S	29.438	1373.078	37.055	0.266	**0.0011**
	Meta-Autoformer-S	76.731	9465.410	97.290	0.693	0.0077
	Meta-FEDformer-S	39.407	2524.772	50.247	0.355	0.0021
Gaussian Filtering	Meta-Transformer-G	28.943	1330.233	36.472	0.261	**0.0010**
	Meta-Informer-G	**28.247**	**1255.989**	**35.440**	**0.255**	**0.0010**
	Meta-Reformer-G	28.694	1294.096	35.974	0.259	**0.0010**
	Meta-Autoformer-G	78.662	10 052.461	100.262	0.710	0.0082
	Meta-FEDformer-G	40.712	2702.249	51.983	0.367	0.0021

4.3. Comparison of heterogeneous algorithms

With the advancement of the Transformer architecture, various improved models based on this framework have emerged. In this section, we will discuss the integration of different large-scale algorithms with meta-learning, following the same parameter settings as mentioned earlier.

Each approach will be examined in detail through experimental discussions.

As shown in Table 4, without filtering, the Meta-Reformer exhibits the best performance. This is primarily because, for the raw data without filtering, Meta-Reformer's local attention mechanism is better at capturing local information. In contrast, Meta-Informer's sparse attention mechanism tends to lose some local information, which results in its poorer performance. Autoformer [35] models long-term dependencies in time series through seasonal decomposition, while FEDformer [12] captures frequency information via frequency domain transformations. However, both Meta-Autoformer and Meta-FEDformer perform worse than the baseline model, Meta-Transformer, with the former showing a dramatic decline in predictive accuracy. This decline can be attributed to the manual decomposition of data, which hinders the internal adaptive decomposition feature of these two algorithms. Since both algorithms rely on decomposition followed by prediction, the lack of effective internal decomposition results in subpar performance on the seasonal and residual components compared to other models. Furthermore, as Meta-FEDformer operates on frequency domain decomposition, the complexity of the frequency components, especially in the residuals, makes it challenging to capture the necessary frequency information, leading to lower prediction accuracy.

After applying adaptive Savitzky-Golay filtering, the prediction errors of Meta-Informer-S and Meta-Reformer-S are nearly identical, with Meta-Informer-S exhibiting slightly higher accuracy. This is because, after filtering, the data becomes smoother, allowing Meta-Informer to effectively learn long-range dependencies through its sparse attention mechanism. However, since the data is smoothed, Meta-Reformer's local attention mechanism becomes restricted. The same observation holds for Gaussian filtering. For Meta-Autoformer, the introduction of adaptive filtering has little to no impact on its prediction accuracy. However, after implementing adaptive filtering, Meta-FEDformer significantly improves its prediction accuracy, primarily due to the smoothing effect of filtering on the residual data, which reduces noise and allows for better frequency component capture in the frequency domain.

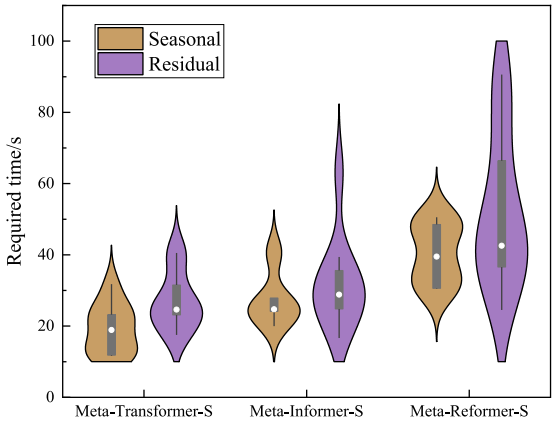


Fig. 12. Comparison of required time for heterogeneous algorithms.

Based on this analysis, the addition of filtering does not substantially affect model run time. Therefore, we will focus our comparative analysis on the models employing Savitzky-Golay filtering, excluding the two models with the highest error rates and conducting ten simulations for each of the remaining three models.

The meta-testing time refers to the duration required for further training to fine-tune the models after the completion of meta-training, utilizing the parameter sets that represent a compromise between the seasonal and residual components. This process not only depends on the model's complexity but also on the quality of the initial training. As shown in Fig. 12, the median time required for Meta-Informer-S and Meta-Transformer-S is nearly identical; however, the time span for the

former is considerably larger. Additionally, the time required for Meta-Reformer-S is higher than that for both preceding models, both in terms of median time and overall time span.

Thus, it can be concluded that the proposed algorithm in this study effectively enhances both prediction accuracy and efficiency.

5. Conclusion

This study proposes a short-term electricity load forecasting algorithm based on sequence decomposition and the Meta-Informer algorithm to enhance prediction accuracy. We integrate the advantages of STL decomposition, clustering, and meta-learning to effectively address the complexities and non-linearities of electricity load data. By using STL decomposition, we break down the electricity load data into trend, seasonal, and residual components, and apply the FLS++ clustering method to group similar patterns in the seasonal and residual components, allowing for optimal utilization of analogous data during prediction. The performance of the Meta-Informer meta-learning model across multiple sub-tasks demonstrates its robust generalization ability and adaptability, particularly in handling complex electricity load data, significantly improving prediction accuracy.

Additionally, we introduce the SDS metric to measure curve smoothness and implement adaptive filtering, effectively enhancing the model's predictive capability for various data patterns while reducing noise interference in the predictions. Furthermore, the BiGRU model exhibits strong performance in forecasting the trend component, ensuring the reliability of the overall prediction results. Experimental results indicate that the proposed framework achieves a favorable balance between prediction accuracy and efficiency, significantly outperforming benchmark forecasting methods. The application of this framework in electricity load forecasting highlights its excellent applicability and robustness, providing substantial support for power management strategies.

CRediT authorship contribution statement

Lianbing Li: Project administration. **Xingchen Guo:** Writing – original draft, Software, Methodology. **Ruixiong Jing:** Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

[1] L. Li, Y. Meng, X. Yuan, J. Li, Peak shaving strategy optimization based on load forecasting: Evidence from Anhui Provence, China, J. Energy Storage 90 (2024) 111818, http://dx.doi.org/10.1016/j.est.2024.111818.

[2] N. Mughees, S.A. Mohsin, A. Mughees, A. Mughees, Deep sequence to sequence Bi-LSTM neural networks for day-ahead peak load forecasting, Expert Syst. Appl. 175 (2021) 114844, http://dx.doi.org/10.1016/j.eswa.2021.114844.

[3] H. Jiang, Y. Dong, Y. Dong, J. Wang, Power load forecasting based on spatial–temporal fusion graph convolution network, Technol. Forecast. Soc. Change 204 (2024) 123435, http://dx.doi.org/10.1016/j.techfore.2024.123435.

[4] G. Juberias, R. Yunta, J. Garcia Moreno, C. Mendivil, A new ARIMA model for hourly load forecasting, in: 1999 IEEE Transmission and Distribution Conference (Cat. No. 99CH36333), vol. 1, 1999, pp. 314–319, http://dx.doi.org/10.1109/TDC.1999.755371.

[5] M.T. Hagan, S.M. Behr, The time series approach to short term load forecasting, IEEE Trans. Power Syst. 2 (1987) 785–791, http://dx.doi.org/10.1109/TPWRS.1987.4335210.

[6] D. Park, M. El-Sharkawi, R. Marks, L. Atlas, M. Damborg, Electric load forecasting using an artificial neural network, IEEE Trans. Power Syst. 6 (1991) 442–449, http://dx.doi.org/10.1109/59.76685.

[7] F. Liu, T. Dong, Q. Liu, Y. Liu, S. Li, Combining fuzzy clustering and improved long short-term memory neural networks for short-term load forecasting, Electr. Power Syst. Res. 226 (2024) 109967, http://dx.doi.org/10.1016/j.epsr.2023.109967.

[8] H. Hua, M. Liu, Y. Li, S. Deng, Q. Wang, An ensemble framework for short-term load forecasting based on parallel CNN and GRU with improved ResNet, Electr. Power Syst. Res. 216 (2023) 109057, http://dx.doi.org/10.1016/j.epsr.2022.109057.

[9] D. Niu, M. Yu, L. Sun, T. Gao, K. Wang, Short-term multi-energy load forecasting for integrated energy systems based on CNN-BiGRU optimized by attention mechanism, Appl. Energy 313 (2022) 118801, http://dx.doi.org/10.1016/j.apenergy.2022.118801.

[10] M. Alhussein, K. Aurangzeb, S.I. Haider, Hybrid CNN-LSTM model for short-term individual household load forecasting, IEEE Access 8 (2020) 180544–180557, http://dx.doi.org/10.1109/ACCESS.2020.3028281.

[11] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, W. Zhang, Informer: Beyond efficient transformer for long sequence time-series forecasting, 2021, arXiv:2012.07436.

[12] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, R. Jin, FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting, 2022, arXiv:2201.12740.

[13] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, M. Long, Itransformer: Inverted transformers are effective for time series forecasting, 2024, arXiv:2310.06625.

[14] Z. Cao, J. Wang, Y. Xia, Combined electricity load-forecasting system based on weighted fuzzy time series and deep neural networks, Eng. Appl. Artif. Intell. 132 (2024) 108375, http://dx.doi.org/10.1016/j.engappai.2024.108375.

[15] G. Zhou, H. Moayedi, M. Bahiraei, Z. Lyu, Employing artificial bee colony and particle swarm techniques for optimizing a neural network in prediction of heating and cooling loads of residential buildings, J. Clean. Prod. 254 (2020) 120082, http://dx.doi.org/10.1016/j.jclepro.2020.120082.

[16] K. Xie, H. Yi, G. Hu, L. Li, Z. Fan, Short-term power load forecasting based on elman neural network with particle swarm optimization, Neurocomputing 416 (2020) 136–142, http://dx.doi.org/10.1016/j.neucom.2019.02.063.

[17] Y. Dai, W. Yu, Short-term power load forecasting based on Seq2Seq model integrating Bayesian optimization, temporal convolutional network and attention, Appl. Soft. Comput. 166 (2024) 112248, http://dx.doi.org/10.1016/j.asoc.2024.112248.

[18] G.-F. Fan, Y.-Y. Han, J.-W. Li, L.-L. Peng, Y.-H. Yeh, W.-C. Hong, A hybrid model for deep learning short-term power load forecasting based on feature extraction statistics techniques, Expert Syst. Appl. 238 (2024) 122012, http://dx.doi.org/10.1016/j.eswa.2023.122012.

[19] Z. Zhang, W.-C. Hong, Application of variational mode decomposition and chaotic grey wolf optimizer with support vector regression for forecasting electric loads, Knowl.-Based Syst. 228 (2021) 107297, http://dx.doi.org/10.1016/j.knosys.2021.107297.

[20] Y. Dai, X. Yang, M. Leng, Forecasting power load: A hybrid forecasting method with intelligent data processing and optimized artificial intelligence, Technol. Forecast. Soc. Change 182 (2022) 121858, http://dx.doi.org/10.1016/j.techfore.2022.121858.

[21] K. Ma, X. Nie, J. Yang, L. Zha, G. Li, H. Li, A power load forecasting method in port based on VMD-ICSS-hybrid neural network, Appl. Energy 377 (2025) 124246, http://dx.doi.org/10.1016/j.apenergy.2024.124246.

[22] H.H. Çevik, M. Çunkaş, K. Polat, A new multistage short-term wind power forecast model using decomposition and artificial intelligence methods, Phys. A 534 (2019) 122177, http://dx.doi.org/10.1016/j.physa.2019.122177.

[23] K. Li, W. Huang, G. Hu, J. Li, Ultra-short term power load forecasting based on CEEMDAN-SE and LSTM neural network, Energy Build. 279 (2023) 112666, http://dx.doi.org/10.1016/j.enbuild.2022.112666.

[24] L. Li, R. Jing, Y. Zhang, L. Wang, L. Zhu, Short-term power load forecasting based on ICEEMDAN-GRA-SVDE-BiGRU and error correction model, IEEE Access 11 (2023) 110060–110074, http://dx.doi.org/10.1109/ACCESS.2023.3322272.

[25] X. Huang, Y. Han, J. Yan, X. Zhou, Hybrid forecasting model of building cooling load based on EMD-LSTM-Markov algorithm, Energy Build. 321 (2024) 114670, http://dx.doi.org/10.1016/j.enbuild.2024.114670.

[26] W. Zha, Y. Ji, C. Liang, Short-term load forecasting method based on secondary decomposition and improved hierarchical clustering, Results Eng. 22 (2024) 101993, http://dx.doi.org/10.1016/j.rineng.2024.101993.

[27] Y. Li, S. Zhang, R. Hu, N. Lu, A meta-learning based distribution system load forecasting model selection framework, Appl. Energy 294 (2021) 116991, http://dx.doi.org/10.1016/j.apenergy.2021.116991.

[28] M. Matijaš, J.A. Suykens, S. Krajcar, Load forecasting using a multivariate meta-learning system, Expert Syst. Appl. 40 (2013) 4427–4437, http://dx.doi.org/10.1016/j.eswa.2013.01.047.

[29] X. Xiao, H. Mo, Y. Zhang, G. Shan, Meta-ANN – A dynamic artificial neural network refined by meta-learning for short-term load forecasting, Energy 246 (2022) 123418, http://dx.doi.org/10.1016/j.energy.2022.123418.

[30] E. Sarmas, E. Spiliotis, E. Stamatopoulos, V. Marinakis, H. Doukas, Short-term photovoltaic power forecasting using meta-learning and numerical weather prediction independent long short-term memory models, Renew. Energy 216 (2023) 118997, http://dx.doi.org/10.1016/j.renene.2023.118997.

[31] R.B. Cleveland, W.S. Cleveland, I. Terpenning, STL: A seasonal-trend decomposition procedure based on loess, J. Off. Stat. 6 (1990).

[32] A. Nichol, J. Achiam, J. Schulman, On first-order meta-learning algorithms, 2018, arXiv:1803.02999.

[33] T. Conrads, L. Drexler, J. Könen, D.R. Schmidt, M. Schmidt, Local search k-means++ with foresight, in: DROPS-IDN/V2/Document/10.4230/LIPIcs.SEA.2024.7, Schloss Dagstuhl – Leibniz-Zentrum Für Informatik, 2024, http://dx.doi.org/10.4230/LIPIcs.SEA.2024.7.

[34] X. Huang, D. Wu, B. Boulet, MetaProbformer for charging load probabilistic forecasting of electric vehicle charging stations, IEEE Trans. Intell. Transp. Syst. 24 (2023) 10445–10455, http://dx.doi.org/10.1109/TITS.2023.3276947.

[35] H. Wu, J. Xu, J. Wang, M. Long, Autoformer: decomposition transformers with auto-correlation for long-term series forecasting, 2022, http://dx.doi.org/10.48550/arXiv.2106.13008, arXiv:2106.13008.